

**LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF320803)
MINGGU 3 ACARA 4**

SEMESTER 3



**Tugas :
PHP Native View, Layout, dan Dasar OOP**

**Disusun Oleh:
Uaryl Engno Laurenca (E41250317)**

**PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026**

A. PENDAHULUAN

Dalam pengembangan aplikasi web, hampir setiap halaman memiliki bagian yang sama, seperti header (logo, menu navigasi) dan footer (informasi copyright). Jika bagian-bagian ini disalin (*copy-paste*) ke setiap halaman satu per satu, maka ketika ada perubahan misalnya menu navigasi bertambah pengembang harus mengedit seluruh halaman satu per satu, yang tidak efisien dan rawan kesalahan.

Untuk mengatasi hal ini, PHP native menyediakan mekanisme *include* dan *require* untuk menyisipkan potongan file (disebut *partial*) ke dalam sebuah layout utama, sehingga bagian yang sama cukup ditulis sekali dan dipakai berulang di banyak halaman. Selain itu, praktikum ini juga memperkenalkan dasar-dasar Object-Oriented Programming (OOP) dalam PHP, yaitu konsep *class*, *object*, *visibility*, *encapsulation*, dan *namespace*, sebagai fondasi sebelum masuk ke pola MVC yang lebih kompleks pada acara-acara berikutnya.

B. TUJUAN PRAKTIKUM

1. Memahami perbedaan dan penggunaan *include*, *require*, *include_once*, dan *require_once*.
2. Membuat struktur layout dan *partial* (*header*, *navbar*, *footer*) agar tidak terjadi duplikasi kode antar halaman.
3. Memahami konsep dasar OOP: *class* sebagai *blueprint* dan *object* sebagai *instance*-nya.
4. Memahami *visibility* (*public*, *private*, *protected*) dan penerapan *encapsulation* melalui *getter/setter*.
5. Memahami kegunaan *namespace* dalam mencegah konflik nama antar *class*.
6. Membuat *class Mahasiswa* sederhana dengan *property private*, kemudian menampilkan data dari beberapa *object*-nya di View menggunakan layout dan *partial* yang sudah dibuat.

C. DASAR TEORI

include dan *require* digunakan untuk menyisipkan file lain ke dalam file yang sedang dijalankan. Perbedaanannya terletak pada penanganan error, yaitu *include* hanya memberikan *warning* dan script tetap berjalan ketika file tidak ditemukan, sedangkan *require* menghasilkan *fatal error* dan menghentikan script. Varian *include_once* dan *require_once* digunakan agar file yang sama hanya dimasukkan satu kali untuk mencegah *redeclaration*. Dalam pengembangan PHP, *require_once* biasanya digunakan untuk file penting seperti konfigurasi dan *class*, sedangkan *include* dapat digunakan untuk bagian halaman yang bersifat opsional.

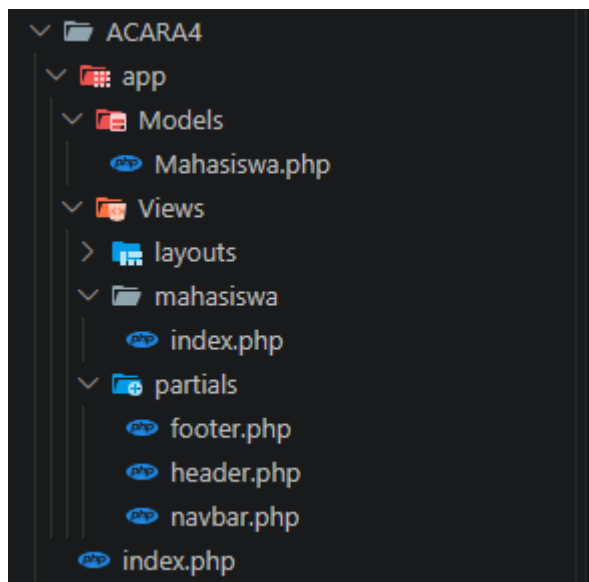
Layout merupakan kerangka utama halaman yang berisi struktur umum seperti `<head>`, *navbar*, konten, dan *footer*, sedangkan *partial* adalah bagian kecil halaman yang dapat digunakan kembali, seperti *header*, *navbar*, atau *footer*. Dengan konsep ini, struktur halaman tidak perlu ditulis berulang-ulang pada setiap halaman. *Class* adalah cetakan yang mendefinisikan *property* dan *method*, sedangkan *object* merupakan bentuk nyata dari *class* tersebut. Dalam PHP, *class Mahasiswa* dapat memiliki *property*

seperti `nim`, `nama`, dan `prodi`, kemudian setiap object yang dibuat dengan `new Mahasiswa()` mewakili data mahasiswa tertentu.

Visibility digunakan untuk mengatur akses terhadap property dan method dalam class, yaitu `public` yang dapat diakses dari mana saja, `private` yang hanya dapat diakses dari dalam class, dan `protected` yang dapat diakses oleh class itu sendiri serta turunannya. Konsep ini berkaitan dengan encapsulation, yaitu menyembunyikan detail internal object dan menyediakan akses melalui method seperti getter dan setter. Sementara itu, namespace digunakan untuk mencegah konflik nama antar class, sehingga class dengan nama yang sama dapat dibedakan berdasarkan namespace-nya, seperti `App\Models\Mahasiswa` dan class lain dengan nama yang sama pada namespace berbeda.

D. HASIL DAN PEMBAHASAN

1. Struktur Folder Layout dan Partial



Dibuat struktur folder `app/Views/` yang memisahkan layout, partial, dan halaman per-modul, mengikuti struktur pada modul praktikum:

2. Implementasi Layout dan Partial

```
app > Views > layouts > main.php
1 <!DOCTYPE html>
2 <html lang="id">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Data Mahasiswa - Acara 4</title>
7   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
8 </head>
9 <body>
10  <?php include __DIR__ . '/../partials/navbar.php'; ?>
11  <main class="container">
12    <?php require $content; ?>
13  </main>
14  <?php include __DIR__ . '/../partials/footer.php'; ?>
15 </body>
16 </html>
```

layouts/main.php berperan sebagai kerangka utama halaman. Partial navbar.php dan footer.php disisipkan menggunakan include, karena sifatnya pelengkap tampilan (jika gagal dimuat, halaman tetap bisa tampil tanpa navbar/footer). Sebaliknya, konten halaman disisipkan menggunakan require \$content, karena konten adalah bagian wajib yang harus ada agar halaman memiliki isi yang bermakna jika file konten tidak ditemukan, aplikasi memang seharusnya berhenti (*fatal error*).

```
app > Views > partials > navbar.php
1  <nav class="navbar navbar-dark bg-dark mb-4">
2      <div class="container">
3          <span class="navbar-brand mb-0 h1">Politeknik Negeri Jember</span>
4      </div>
5  </nav>
```

```
app > Views > partials > footer.php
1  <footer class="text-center text-muted mt-5 py-3 border-top">
2      &copy; <?= date('Y') ?> Politeknik Negeri Jember – Workshop SI Web Server
3  </footer>
```

3. Implementasi Class Mahasiswa

```
app > Models > Mahasiswa.php
1  <?php
2
3  namespace App\Models;
4
5  class Mahasiswa
6  {
7      private string $nim;
8      private string $nama;
9      private string $prodi;
10
11     public function __construct(string $nim, string $nama, string $prodi)
12     {
13         $this->nim = $nim;
14         $this->nama = $nama;
15         $this->prodi = $prodi;
16     }
17
18     public function getNim(): string
19     {
20         return $this->nim;
21     }
22
23     public function getNama(): string
24     {
25         return $this->nama;
26     }
27
28     public function getProdi(): string
29     {
30         return $this->prodi;
31     }
32
33     public function setNama(string $nama): void
34     {
35         if (strlen($nama) < 3) {
36             throw new \InvalidArgumentException("Nama terlalu pendek");
37         }
38         $this->nama = $nama;
39     }
40
41     public function getLabel(): string
42     {
43         return "{$this->nim} - {$this->nama} ({$this->prodi})";
44     }
45
46     // Tugas Mandiri: angkatan dari 2 digit awal NIM (misal '24' -> '2024')
47     public function getAngkatan(): string
48     {
49         $duaDigit = substr($this->nim, 0, 2);
50         return '20' . $duaDigit;
51     }
52 }
```

Dibuat class Mahasiswa pada namespace App\Models dengan tiga property private (nim, nama, prodi), constructor untuk inisialisasi ketiga property tersebut, serta getter (getNim(), getNama(), getProdi()) untuk mengakses nilainya dari luar class. Method setName() dilengkapi validasi sederhana yang melempar InvalidArgumentException apabila nama yang dimasukkan kurang dari 3 karakter, sebagai penerapan *encapsulation*. Ditambahkan pula method getLabel() yang menggabungkan NIM, nama, dan prodi menjadi satu string deskriptif, sesuai contoh pada modul.

4. Pembuatan Object dan Pengiriman Data ke View

```
index.php
1 <?php
2
3 // Autoloader sederhana untuk namespace App\...
4 spl_autoload_register(function ($class) {
5     $class = str_replace('App\\', '', $class);
6     $file = __DIR__ . '/app/' . str_replace('\\', '/', $class) . '.php';
7     if (file_exists($file)) {
8         require_once $file;
9     }
10 });
11
12 use App\Models\Mahasiswa;
13
14 // Membuat beberapa object Mahasiswa
15 $mahasiswaList = [
16     new Mahasiswa('2401001', 'Budi Santoso', 'Teknik Informatika'),
17     new Mahasiswa('2301002', 'Siti Aminah', 'Sistem Informasi'),
18     new Mahasiswa('2201003', 'Andi Wijaya', 'Manajemen Informatika'),
19 ];
20
21 // Kirim ke View lewat layout
22 $content = __DIR__ . '/app/Views/mahasiswa/index.php';
23 require __DIR__ . '/app/Views/layouts/main.php';
```

Pada file index.php (sebagai entry point sementara, sesuai instruksi modul), dibuat beberapa object Mahasiswa menggunakan new Mahasiswa(...) dengan data NIM, nama, dan prodi yang berbeda-beda, kemudian dikumpulkan ke dalam sebuah array. Array tersebut dikirim ke View melalui variabel yang tetap berada dalam scope saat layout dan konten di-require.

5. Menampilkan Data di View dengan Tabel Bootstrap

Pada Views/mahasiswa/index.php, data dari setiap object Mahasiswa ditampilkan dalam tabel Bootstrap menggunakan perulangan foreach, dengan memanggil method getter (\$mhs->getNim(), \$mhs->getNama(), \$mhs->getProdi(), \$mhs->getLabel()) — bukan mengakses property secara langsung, karena property sudah bersifat private sesuai prinsip *encapsulation*.

6. Tugas Mandiri Method Angkatan dari NIM

Ditambahkan method getAngkatan() pada class Mahasiswa yang mengambil 2 digit pertama dari NIM menggunakan substr(\$this->nim, 0, 2), kemudian menggabungkannya dengan prefix '20' untuk menghasilkan tahun angkatan (misalnya NIM 2401001 menghasilkan angkatan 2024). Hasil method ini ditambahkan sebagai kolom baru pada tabel di View, sehingga setiap baris data mahasiswa juga menampilkan angkatannya.

Data Mahasiswa

NIM	Nama	Prodi	Angkatan	Label
2401001	Budi Santoso	Teknik Informatika	2024	2401001 - Budi Santoso (Teknik Informatika)
2301002	Siti Aminah	Sistem Informasi	2023	2301002 - Siti Aminah (Sistem Informasi)
2201003	Andi Wijaya	Manajemen Informatika	2022	2201003 - Andi Wijaya (Manajemen Informatika)

E. KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa penggunaan `include` dan `require` sesuai fungsinya dapat membuat aplikasi lebih aman dan fleksibel. Pemisahan layout dan partial seperti `header`, `navbar`, dan `footer` juga berhasil mengurangi duplikasi kode sehingga perubahan tampilan cukup dilakukan pada satu file. Class `Mahasiswa` berhasil menerapkan konsep dasar OOP melalui *encapsulation*, constructor, serta getter dan setter untuk mengatur akses data. Penggunaan *namespace* `App\Models` memberikan struktur penamaan yang lebih jelas dan mencegah konflik nama class. Selain itu, data dari beberapa object `Mahasiswa` dapat ditampilkan melalui View menggunakan layout, partial, dan method getter tanpa perlu mengetahui detail internal Model. Pembuatan method `getAngkatan()` juga menunjukkan bahwa logika pengolahan data seperti mengambil tahun angkatan dari NIM sebaiknya ditempatkan di dalam Model agar tanggung jawab antara Model dan View tetap terpisah dengan baik.